Project - High Level Design on Dockerized Sports Python Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Team Members:

Sr no	Student Name	Enrolment Number
1	Priyanshi Gandhe	EN22CS301764
2	Priyanshu Sengar	EN22CS301770
3	Rudra Yadav	EN22CS301831
4	Prithviraj Singh Tomar	EN22CS301757
5	Pranjali Sharma	EN22CS301733

Table of Contents

1.Introduction

1.1 Scope of the document
1.2 Intended Audience
1.3 System Overview

2.System Design

2.1 Application Design
2.2 Process Flow
2.3 Information Flow
2.4 Components Design
2.5 Key Design Considerations
2.6 API Catalogue

3. Data Design

3.1 Data Model
3.2 Data Access Mechanism
3.3 Data Retention Policies
3.4 Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirement

7.1 Security Aspects
7.2 Performance Aspects

8. Reference

HLD on Dockerized Sports Python Service

1. Introduction

1.1 Scope of the Document

This document describes the **high-level architecture and design of the Sportlytics application**, an AI-powered sports analysis platform built using Python. It explains the system architecture, application components, data flow, APIs, and deployment strategy using **Docker and containerized environments**.

1.2 Intended Audience

- Developers
- DevOps Engineers
- Project Reviewers / Faculty Evaluators
- System Architects

1.3 System Overview

Sportlytics is an **AI-powered sports analytics platform** that allows users to query sports-related insights and receive detailed analysis reports.

The system is built using:

- **Streamlit** for the web interface
- **CrewAI multi-agent system** for intelligent reasoning
- **Groq LLM** for fast AI inference
- **Tavily Search API** for real-time sports information
- **SQLite database** for storing users and history
- **Docker** for containerized deployment

The platform processes a user's sports query, coordinates multiple AI agents to analyze the query, gathers external information, and generates a structured analytical report.

2. System Design

2.1 Application Design

Technology Stack

- Frontend: Streamlit
- Backend: Python
- AI Framework: CrewAI Agents
- LLM Provider: Groq
- Search API: Tavily
- Database: SQLite
- Containerization: Docker (multi-stage builds)

Architecture Type

Modular monolithic application with AI agent orchestration.

Application Layers

1. **UI Layer**
    - Streamlit interface
    - Login / Signup system
    - Query submission
    - Report display
2. **AI Processing Layer**
    - Planner Agent
    - Analyst Agent
    - Reporter Agent
3. **Integration Layer**
    - Groq LLM API
    - Tavily Search API
4. **Data Layer**
    - SQLite database
    - User authentication
    - Query history storage

2.2 Process Flow

1. User logs into the Sportlytics platform
2. User submits a sports analysis query
3. Query is processed by the **Planner Agent**
4. Planner Agent divides the task into smaller subtasks
5. **Analyst Agent** collects relevant sports data using Tavily Search
6. Data is processed using **Groq LLM** for reasoning
7. **Reporter Agent** generates a structured analysis report
8. Report is displayed on the Streamlit interface
9. Query and response are stored in the SQLite database

2.3 Information Flow

System data flow:

User
→ Streamlit UI
→ CrewAI Agents
→ Groq LLM API
→ Tavily Search API
→ AI Processing
→ Report Generation
→ SQLite Database Storage
→ Response back to User

2.4 Components Design

1. Streamlit Web Interface

Provides an interactive UI for:

- User authentication
- Query submission
- Displaying sports analysis reports

2. AI Agents (CrewAI)

Planner Agent

- Breaks user queries into tasks

Analyst Agent

- Searches external sources using Tavily
- Collects sports data

Reporter Agent

- Converts analysis into readable reports

3. Database (SQLite)

Stores:

- User accounts
- Query history
- Generated reports

4. External APIs

Groq API

- Used for high-speed AI inference

Tavily API

- Used for retrieving sports-related information from the web

5. Docker Container

- Packages the entire application
- Ensures portability and consistent deployment

2.5 Key Design Considerations

Scalability

- Docker-based architecture
- Future Kubernetes compatibility

Security

- Password hashing (SHA-256)
- Environment variables for API keys

Performance

- Groq provides extremely fast LLM inference
- Lightweight SQLite database

Portability

- Fully containerized deployment using Docker

2.6 API Catalogue

Internal APIs used by the application:

Method	Endpoint	Description
POST	`/login`	User authentication
POST	`/signup`	Register new user
POST	`/query`	Submit sports analysis query
GET	`/history`	Retrieve previous queries

External APIs:

API	Purpose
Groq LLM API	AI reasoning and report generation
Tavily Search API	Real-time sports information retrieval

3. Data Design

3.1 Data Model

Users Table

Field	Type	Description
id	Integer	Primary Key
username	String	Unique username
password	String	Hashed password

History Table

Field	Type	Description
id	Integer	Primary Key
user_id	Integer	Linked to user
query	Text	User query
response	Text	Generated report
timestamp	DateTime	Query time

3.2 Data Access Mechanism

- Python **sqlite3 module**
- Direct queries from the application
- Simple CRUD operations

3.3 Data Retention Policies

- User queries stored in **history table**
- Data remains stored unless manually deleted
- Used for tracking previous analyses

3.4 Data Migration

- SQLite database stored as a **local database file**
- Persistent storage when deployed using **Docker volumes or Kubernetes PVCs**

4. Interfaces

Web Interface

- Built using **Streamlit**
- Responsive layout
- Custom CSS styling

Developer Interface

- Docker CLI
- Docker Compose for container orchestration

5. State and Session Management

Sportlytics manages user sessions using **Streamlit Session State**.

Functions include:

- Maintaining login status

- Multi-page navigation
- Temporary storage of user queries during sessions

---

6. Caching

Currently minimal caching is implemented.

Possible caching mechanisms:

- Streamlit caching (`st.cache_data`)
- Storing frequently accessed results

This can improve performance in future versions.

7. Non-Functional Requirements

7.1 Security Aspects

- Password hashing using SHA-256
- API keys stored in **environment variables**
- Docker containers run with **non-root user**
- Secrets managed through `.env` files

7.2 Performance Aspects

- Groq LLM provides **fast inference speeds**
- Lightweight SQLite database
- Optimized **multi-stage Docker build**
- Efficient multi-agent task execution

8. References

Documentation references:

- Python Documentation
- Streamlit Documentation
- CrewAI Documentation
- Groq API Documentation
- Tavily API Documentation
- Docker Documentation